CHAPTER 10: ENTRY POINTS, ROUTES, AND THE INTEGRATION LAYER

Summary: *Three questions live here and they carry more exam weight than anything else in the book. Which capabilities an agent should hold and when their definitions load. Where authorisation is actually enforced, as opposed to where it appears to be. And which integration mechanism fits, given who owns the system and whether a live call is needed at run time. Each has a precise answer, and in each case the wrong answers are competent-sounding measures applied one layer too late.*

Capability Bloat Is a Loading Problem, Not a Count Problem

Two questions get confused constantly, and separating them settles several items at once.

Should this tool exist at all? A tool stays because you can point at real use of it, either by some named piece of work, or a handoff that happens on a schedule. A **plausible imagined use is not a reason to keep a tool**. “Somebody might need to query that” is a hypothesis, not a workload, and it costs context on every request forever.

When should its definition enter the context? This is a different question with a different fix, and it is the one people miss. Definitions can be deferred so they are not sent to the model up front, and loaded when a task needs them.[1] That change **drops what you pay per request** without **taking a single capability away**. Nothing is removed. The agent can still do everything it could before.

So when a scenario complains about context cost from a large tool catalogue, ask whether the option you are looking at changes *how many tools exist* or *when their definitions arrive*. The second one is usually the answer, because it is the only one that does not amputate capability to save tokens.

There is a third, separate symptom that needs a third answer. When the model picks the **wrong** tool, the fix is neither of the above: it is reducing the **number of near-identical options** the model is asked to **choose between** on any one request. Adding context does not help. Adding per-call work does not help. Swapping in a

stronger model does not help, because the catalogue is unchanged and the ambiguity is in the catalogue.

Two Changes Fix Tool Ambiguity, and Three Make It Worse

When tools are being confused with each other, the cause is worth naming precisely: **two entries really do overlap in the job they perform**, rather than being described too briefly.

That distinction determines which fixes work. Two changes, and only two, reach the ambiguity itself:

Make each surviving boundary explicit. Say what each tool is for and, where two are close, what separates them. Clear, specific descriptions are documented as improving selection accuracy across large tool sets.[1]

Delete the overlaps outright. If two tools genuinely do the same job, one of them is a distractor you are paying for on every request.

Three tempting alternatives make things worse or leave them unchanged. Adding another tool to handle the confusing case adds a candidate. Post-processing the wrong choice masks the symptom while the selection stays wrong. And replacing typed tool definitions with prose instructions trades a checkable schema for something nothing can validate.

Where the Context Cost Actually Comes From

One diagnostic rule saves you from a family of plausible wrong answers.

Context cost comes from what the **session assembles into the request payload** before the **first turn**. Tool definitions, system prompt, loaded files, injected instructions: things that become part of what is sent.

Everything else named in these scenarios is either presentation-layer or imagined. How a client displays results does not reach the payload. Hidden standing state that the model supposedly carries between sessions does not exist and therefore occupies nothing. If it does **not get converted into request content, it costs no context**, however plausible the mechanism sounds.

Authorisation Belongs Outside the Model

Now the most important idea in this chapter, and the one that decides the most items.

The question is always this: which layer here is the one that **genuinely settles what a request is able to hand back**?

Here is the defective design that keeps appearing in different costumes. The retrieval path can reach everyone's records. The system prompt tells the model to answer only about the current user's account. It works when you test it.

It is broken, and the reason is not that the instruction is weak. The reason is that **one lapse in following instructions** stops being a **bad answer** and becomes **other people's records leaving the building**. The blast radius of a prompt failure is set by what the layer underneath permits, and here the layer underneath permits everything.

So the correct design has a grant **sized to match the job the agent was given**, and **enforced somewhere the model cannot reach**. That formulation excludes two failure modes at once, which is why it keeps being the right answer:

Over-grants that lean on model restraint. The permission exists, and the only thing stopping its use is instruction.

Under-grants that break the stated function. Locked down past the point where the agent can do its job, which gets reversed under pressure and usually reversed too far.

The **named anti-pattern is the prompt as an access control**. It is seductive because it works in every demo.

Place the Control Where Identity Is Known and Content Can Still Be Withheld

The placement rule follows from the same idea and is worth memorising as a sentence.

Put the control at the **first moment you know who is asking** and can **still decline to hand anything** over.

Everything later is scrubbing data that has already crossed the trust boundary. Filtering after retrieval means the records were fetched, existed in memory, and probably passed through a log. Redacting in the response means the sensitive value was assembled into a payload first.

Two specific placements are findings rather than designs, and a scenario may ask you to identify both:

A secret placed inside the prompt. It has crossed into a context you do not control, and it will appear in logs, in traces, and possibly in output.

An entitlement checked after retrieval. The data was already read on behalf of somebody who may not have been allowed to read it.

Both share the shape: a control positioned after the material moved. Controls that keep the boundary intact are not findings, however unglamorous they look.

Three Layers, So One Mistake Is Never Enough

For agents that can change things, the documented shape of a safe configuration clamps down in three separate places at once, on the identity, on what is exposed, and on what needs a human:

Layer	What it constrains	What it prevents alone
The credential	What the agent's identity is permitted to do in the target system	Anything outside the grant, regardless of what the agent tries
The tool surface	Which operations are exposed to the model at all	Calls that were never available to be made
The approval gate	Which actions require a human before they commit	Unreviewed state changes

The point of all three is not redundancy for its own sake. It is that getting any **one of them wrong** should **never be sufficient** to let an **unreviewed write reach production**. One layer misconfigured is a near miss. One layer total is an outage waiting for a bad afternoon.

The connector documentation supports this directly: individual tools can be enabled and disabled with allowlist and denylist patterns, and the recommendation is explicit that **write or destructive tools** should be **denied** when building a **read-only assistant**, or when you want a **human confirmation step before state changes**.[1]

Choosing the Integration Mechanism

Three mechanisms, and one question that separates the first two.

At the moment this capability runs, does it have to **reach out and talk to another system**?

If yes, it is MCP-shaped. If what you are packaging is know-how, a procedure somebody follows, and nothing has to be fetched while it runs, that is **Skill-shaped**. Nearly every distractor in this area either points at MCP or names something both mechanisms satisfy equally, which makes the live-call question the only one that discriminates.

When a scenario asks which factors actually bear on the choice, only two describe the **integration rather than the people building** it:

Whether more than one client will use it.

Whether the exchange carries state between calls or does not.

Everything else on those lists describes the implementing team, their skills, their preferences, their timeline, or names a requirement every candidate already meets. A **fact about the team is not a property of the integration**.

Then the combination that has exactly one answer: when **ownership is spread out**, **capabilities keep changing**, and **several applications consume them**, the only arrangement that survives is **one integration built and maintained by whoever owns the system**, which everybody else simply **calls**. Point-to-point integrations fail that test on all three counts, because twelve consuming teams cannot absorb changes to twelve systems they do not own.

Agent-to-agent has its own discriminating property, and it is a trust-boundary one. The correct arrangement keeps **every organisation's agent** as the **only thing that ever touches that organisation's systems**, so what travels between them is negotiation and nothing else. Not tools. Not data. An arrangement where one side publishes its internal tools to the other, or where a third party is trusted with both sides' systems, has moved the boundary rather than crossed it carefully.

MCP: What the Protocol Guarantees, and What It Does Not

The Model Context Protocol is an open protocol connecting language model applications to external data and tools, using JSON-RPC 2.0 messages between hosts that initiate connections, clients inside those hosts, and servers that provide context and capabilities. Servers offer resources, prompts and tools, and the specification compares its role to how the Language Server Protocol standardised language support across editors.[2]

The architecture carries a security property worth knowing exactly, because scenarios are built on it. The **host** controls **connection permissions**, enforces **security policies and consent requirements**, and handles **user authorisation decisions**. And as a stated design principle, servers should **not be able to read the whole conversation** or **see into other servers**, with **full conversation history staying with the host**.[3]

Two operational facts, both of which change designs:

The **connector is not covered by zero-data-retention arrangements**. Data exchanged with servers, including tool definitions and execution results, is retained under the standard retention policy.[1] If a scenario has a zero-retention requirement, that is a hard constraint on this route.

Tunnels reach servers inside a private network over an outbound-only connection, which removes the need to open inbound firewall ports, expose services publicly, or allowlist address ranges at the origin.[4] When a scenario frames the choice as "expose it or allowlist us", that is a false pair.

And the governance fact, which is a live risk in any organisation: by default, **any user running Claude Code can connect any MCP server** they choose. Anthropic reviews

connectors against its listing criteria before adding them to its directory but does **not security-audit or manage any MCP server**. Administrators can restrict this across a documented range: disabling servers entirely, deploying a fixed managed set users cannot add to, publishing an approved catalogue, or leaving users unrestricted.[5]

Tokens, Scopes, and the One Explicitly Forbidden Pattern

Authorisation in the specification is optional for implementations, but where it applies the rules are strict.[6] HTTP-transport implementations should conform to it, while implementations using standard input and output should take credentials from the environment instead. And the audience rule: servers must **validate that access tokens were issued specifically for them**, must only **accept tokens valid for their own resources**, and must **not accept or transit any others**.

That last clause is the foundation for the one pattern the security guidance names as explicitly forbidden.[7]

Token passthrough is forbidden. Its documented consequences are worth carrying as three separate harms, because a scenario may test any of them:

- Controls that depend on token audience are circumvented, since the token is being used somewhere it was not issued for.
- Downstream logs show requests appearing to come from a different identity, which destroys attribution exactly when you need it.
- The server becomes a proxy for exfiltration wherever claims are not validated.

The same document covers the confused deputy problem, and a scope minimisation section whose failure modes are worth naming. Broad up-front scopes expand the blast radius, make revocation disruptive to all workflows at once, and cause consent abandonment, where users stop reading requests they cannot evaluate. It also names scope inflation blindness, where an absence of metrics quietly normalises over-broad requests.

The recommended mitigation is specific: a **minimal initial scope set, targeted incremental elevation, and logged elevation events**. Note that all three are required. Minimal scopes with no elevation path break the function; elevation without logging removes the evidence.

Where Claude Runs Changes the Contract

Deployment route is an integration decision with documented, testable differences.

Route	Documented differences that matter
Amazon Bedrock	Runs on AWS-managed infrastructure with **zero operator access**, with Anthropic personnel having no access to the inference infrastructure. Model identifiers carry a provider prefix. **Current computer and browser tool sets are not available**, though earlier beta versions remain.[8]
Google Cloud	The model is named in the **endpoint URL** rather than in the **request body**, and a **version field goes in the body rather than in a header**. **Deprecation and retirement terms are set by the partner** and can differ from the Claude API schedule.[9]

Route	Documented differences that matter
Microsoft Foundry	**Two hosting options chosen at deployment configuration time**, one on Azure infrastructure and one on Anthropic infrastructure, differing in which models and deployment types are available. Billed through the Azure Marketplace, and a **deployment name cannot be changed after creation**.[10]

Two of those rows are the kind of detail that decides an item. A **partner-operated platform sets its own lifecycle schedule**, so a migration plan built on the Claude API's retirement dates does not transfer. And a **deployment name that cannot be changed after creation** is an irreversible decision hiding inside a configuration screen.

Finally, the identity mechanics.[11] Three authentication methods are documented: API keys, Workload Identity Federation for workloads that already hold a platform-issued identity, and App Attest for iOS and macOS applications distributed to end users. Keys can carry preset or custom expirations, and the guidance is blunt about what that buys: **expiration limits the lifetime of a leaked credential** and is **not a substitute for secret hygiene**.

Workspaces separate projects, environments or teams inside one organisation while keeping billing and administration central, with API keys, members and resource limits assignable per workspace and members able to hold different roles in each.[12] When a scenario needs environment separation without organisational separation, that is the mechanism.

How Wrong Answers Are Built on Integration and Access Questions

- A **control placed after the boundary**. The option filters, redacts or checks entitlement once the data has already been fetched or the secret already

embedded. The test: at the moment this control runs, has the **sensitive material already moved**?

- **Instruction standing in for enforcement**. The option narrows behaviour with a prompt while the underlying permission stays broad. The test: if the model **ignored this instruction entirely**, what could the **request return**?
- **Capability removed to save context**. The option deletes tools rather than deferring their definitions. The test: does this reduce what the **agent** can do, or only what is **loaded up front**?
- A **property of the team offered as a property of the integration**. The test: would this fact **still be true if a different team built** it?
- **One layer where three are needed**. The test: is there a **single misconfiguration** here that permits an **unreviewed write**?

When the scenario says	The answer is usually	The trap is usually
A large tool catalogue is consuming context	Defer definitions so they load when a task needs them	Deleting tools, which removes capability to save tokens
A tool might be useful someday	Remove it; only real use, by named work or a recurring handoff, earns a place	Keeping it against an imagined use
The model chooses the wrong tool	Cut how many near-identical options it faces per request	More context, more per-call work, or a stronger model

When the scenario says	The answer is usually	The trap is usually
Tools overlap, two answers wanted	Make each remaining boundary explicit and delete genuine duplicates	Adding a tool for the confusing case, post-processing the choice, or replacing schemas with prose
Something is blamed for context cost	Only what the session assembles into the payload before the first turn	Presentation-layer behaviour or imagined standing state
A prompt restricts what the model may disclose, while retrieval can reach everything	Enforce the restriction below the model, outside its control	Strengthening the instruction
Access must be limited	A grant sized to the job the agent was given, enforced beyond the model's reach	Over-granting and relying on restraint, or under-granting until the function breaks
You must choose where a control sits	The first moment you know the caller and can still refuse to hand anything over	Post-retrieval filtering or response redaction
An agent can write to production	Clamp the identity's permissions, the exposed operations, and the human approval step	Relying on any single one of the three

When the scenario says	The answer is usually	The trap is usually
The capability must reach another system while it runs	MCP	A Skill, which carries know-how and fetches nothing
The capability is know-how that fetches nothing at run time	A Skill	MCP, which is more machinery than the job needs
Which factors bear on the integration choice	How many clients will call it, and whether state carries between calls	Team skills, timelines, preferences, or requirements every option meets
Ownership spread out, capabilities changing, many consumers	One integration built by the system's owners and called by all	Point-to-point integrations maintained by consumers
Two organisations' agents must cooperate	Each side's agent alone touches that side's systems; only negotiation travels	Publishing internal tools outward, or trusting a third party with both sides
Zero data retention is required	The connector route does not qualify	Assuming retention policy follows the rest of the account
A private-network server must be reachable	An outbound-only tunnel	Opening inbound ports or allowlisting address ranges

When the scenario says	The answer is usually	The trap is usually
A token is forwarded to a downstream service	Forbidden; validate audience and issue your own	Passing it through because it works
Broad scopes are requested up front	Minimal initial scope, targeted elevation, logged elevation events	Requesting everything once to avoid interruptions later
Deployment is on a partner platform	Check the partner's own lifecycle schedule	Assuming Claude API retirement dates apply
Environments must be separated inside one organisation	Workspaces, with per-workspace keys, members and limits	A separate organisation, or shared keys with naming conventions

The habit worth carrying: for every control in a design, ask **what it protects** if the model **does exactly the wrong thing**. Controls that only work when the model behaves are not controls.

What to Remember

- Keeping a tool requires **some named piece of work**, or a **handoff that happens on a schedule**. An **imagined future use is not a justification**, and it costs context on every request.
- **Deferring when tool definitions load** is a different fix from **deleting tools**. Deferral cuts the per-request bill **without removing anything the agent** can do. Deleting tools **removes capability to save tokens**.
- The model picks the wrong tool because **two entries genuinely cover the same ground**, not because their descriptions run short. Two things fix it: **making each remaining boundary explicit**, and **deleting real duplicates**. Adding a tool, post-processing the choice, or swapping schemas for prose all fail.

- Context cost comes only from what the session **assembles into the request payload** before the first turn. **Presentation-layer behaviour** and **imagined standing state** reach nothing.

- Always ask which **layer genuinely settles** what a request is able to hand back. A retrieval path that can reach everyone's data, narrowed only by a prompt, turns a **single lapse in following instructions** into **records leaving the building**, rather than into a bad answer.

- The right grant is **sized to the job the agent was actually given**, and **enforced somewhere the model cannot reach**. That excludes **over-grants relying on model restraint** and **under-grants that break the stated function**.

- Put every control at the **first moment you know who is asking** and can **still refuse to hand anything** over. Everything later **scrubs data that already crossed the boundary**. A secret inside the prompt and an entitlement checked after retrieval are **both findings**, because both sit **after the material moved**.

- An agent that can write is constrained in **three separate places**: what its **identity** may do, which **operations are exposed to it at all**, and which **actions need a human** before they commit. Get any single one of them wrong and that still must **not be enough** to let an unreviewed write reach production. Deny write and destructive tools for read-only assistants or where human confirmation is wanted.

- The mechanism question is whether the thing has to **talk to another system while it runs**. If it does, that is **MCP**. Know-how that **fetches nothing at run time** is **Skill-shaped**.

- Only **two of the usual factors** describe the integration rather than its builders: **whether more than one client will use** it, and **whether the exchange carries state between calls**. Team skills, preferences and timelines are **facts about the builders**.

- **Spread-out ownership**, **capabilities that keep changing**, and **several consuming applications** together force **one integration**, built by **whoever owns the system** and simply called by everybody else, because consuming teams cannot absorb changes to systems they do not own.
- In agent-to-agent work, **each side's agent** is the **only thing that ever touches that side's systems**, so **negotiation is all that travels** between them, never tools and never data.
- MCP uses **JSON-RPC 2.0** between **hosts, clients and servers**, servers offering resources, prompts and tools. The **host controls permissions**, security policy, consent and authorisation, and by design servers **cannot read the whole conversation** or **see into other servers**, with full history staying with the host.
- The **connector is not covered by zero-data-retention**, and data exchanged with servers including tool definitions and execution results is **retained under the standard policy**.
- Tunnels reach private-network servers over an **outbound-only connection**, removing any need to **open inbound ports**, expose services publicly, or allowlist address ranges.
- By default **any Claude Code user can connect any MCP server**. Anthropic reviews connectors for directory listing but does **not security-audit or manage any server**; administrators can **disable servers**, **deploy a fixed managed set**, **publish an approved catalogue**, or leave users unrestricted.
- Servers must **validate that a token was issued** for them, accept only tokens valid for their **own resources**, and **never accept or transit others**. **Token passthrough is explicitly forbidden**, and its harms are **circumvented audience controls**, **downstream logs showing the wrong identity**, and the **server acting as an exfiltration proxy**.
- Broad up-front scopes **expand blast radius**, **make revocation disruptive**, **cause consent abandonment**, and produce **scope inflation blindness** when

nothing is measured. The mitigation is **minimal initial scopes**, **targeted incremental elevation**, and **logged elevation events**, all three.

- **Deployment route** changes the contract. Bedrock runs with **zero operator access** and lacks the current computer and browser tools. Google Cloud puts the **model in the URL and the version in the body**, and partner platforms set their **own deprecation schedules**. Microsoft Foundry picks **Azure or Anthropic infrastructure** at deployment configuration time, and its **deployment name is immutable after creation**.

- Authentication comes in **three documented forms**: **API keys**, **Workload Identity Federation** for workloads holding a platform-issued identity, and **App Attest** for distributed iOS and macOS apps. **Key expiration limits a leaked credential's lifetime** and does **not replace secret hygiene**.

- **Workspaces separate projects, environments or teams** within one organisation while **billing and administration stay central**, with keys, members and resource limits assignable per workspace.